Testen

1. Blinkende Leuchtdiode (LED)

In diesem ersten Versuch werden wir ein Programm schreiben, dass eine Leuchtdiode zum Blinken bringt. Hierzu verwenden wir das Software-Entwicklungstool „Arduino-IDE" und die Bauteile des Workshops „Testen“. Machen Sie sich mit den zur Verfügung gestellten Bauteilen vertraut und bauen Sie die Schaltung nach Abbildung 1.1 auf. Wenn Sie sich unsicher sind fragen Sie einen der anwesenden Betreuer um Rat.

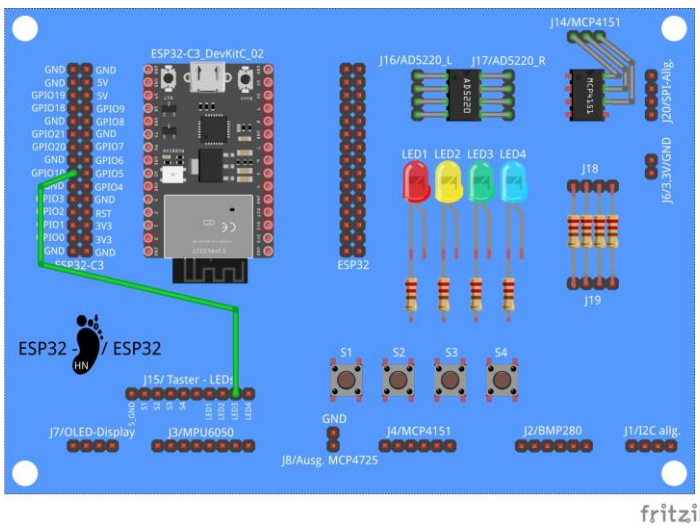


Abbildung 0-1.1:Verdrahtungsplan für die Ansteuerung einer LED

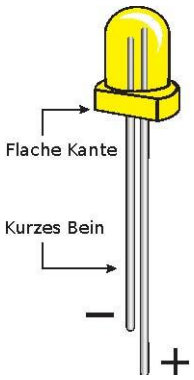
Zum Verbinden der Bauteile nutzen Sie sogenannte Jumper Kabel (oder auch Steckbrücken genannt). Die Farben können dem Entwickler bei der Orientierung helfen und sollten wenn möglich wie gezeichnet verwendet werden. Die Farbe hat allerdings keinen Einfluss auf elektrische Eigenschaften.

Auf Ihrer Platine sind die LED’s und Widerstände bereits fest verlötet. Eine Seite der Widerstände ist dabei mit der Masse (GND) des Mikrocontrollers (ESP32-C3) verbunden, die andere Seite mit der Kathode (kurzes Bein) der LED. Die Eingangsseite (Anode – langes Bein) der LED ist mit der Buchsenleiste J15 verbunden, wie Sie der Abbildung 1.2 entnehmen können.



Abbildung 1.0-2 Buchsenleiste J15

Testen



Nochmals zur Erläuterung:
Wie Sie aus dem Lötworkshop möglicherweise bereits wissen, ist bei Verwendung einer LED auf die richtige Polarität zu achten. Das kürzere Beinchen und die flache Kante zeigen den negativen Pol der LED an.

Der verwendete LED-Vorwiderstand hat einen Wert von 330 Ω und dient zur Begrenzung des Stroms.

Nun werden wir gemeinsam den ESP32-C3 programmieren.

Die Bedienung der Arduino IDE[1] wird auf den Bildschirmen und/oder auf dem Beamer vom Dozenten gezeigt. Beachten Sie insbesondere die Einstellungen für die Platine (Menü Werkzeuge / Board -„ESP32C3 Dev Module“) und die Auswahl des richtigen COM-Ports (Menü Werkzeuge / Port).

Folgendes Programm wird erstellt:

```
/* Blink: Eine Leuchtdiode wird periodisch ein-und ausgeschaltet. */

int gruen = 10; // Port der gruenen LED definieren

void setup()
{                                   // Funktion setup() wird 1x am Anfang ausgeführt
        pinMode(gruen, OUTPUT);     // Port 10 wird zum Ausgang definiert
}

void loop()
{                                   // Funktion loop() wird ständig wiederholt
        digitalWrite(gruen, HIGH);  // Port 3 wird auf 5V geschaltet
        delay(1000);                // eine Sekunde warten (1000 ms = 1 s)
        digitalWrite(gruen, LOW);   // Port 3 wird auf 0V geschaltet
        delay(1000);                // eine Sekunde warten
}
```

Nach Eingabe des Programms können Sie es durch einen Klick auf die Schaltfläche „Hochladen“ (der Pfeil →) auf den ESP32-C3 überspielen[2]. Nach dem erfolgreichen Upload sollte die LED im Sekundentakt umschalten. Sollte dies nicht funktionieren, so sprechen Sie bitte einen Betreuer an.

Testen

[1] IDE: Integrated Development Environment (zu deutsch: Integrierte Entwicklungsumgebung)

[2] Manchmal kann es beim Hochladen des Programmes zu Verbindungsfehlern kommen. In diesem Fall kann es helfen den „Boot“ Button links neben dem USB-Anschluss zu drücken, wenn in der Arduino-IDE im unteren Fenster die Funktion „Connecting …..“ erscheint.
Wenn Sie mehr über den Kompilierungsvorgang erfahren möchten schalten Sie in der Arduino-IDE unter „Voreinstellungen“ die „Ausführliche Ausgabe“ für Kompilierung ein.

Aufgaben

- Ändern Sie in Ihrem Programm die Blinkfrequenz. Testen Sie, ob es möglich ist den Zustand (High/Low) so schnell zu wechseln, dass dies mit dem bloßen Auge nicht mehr sichtbar ist.

- Erweitern Sie Ihre Schaltung um eine gelbe und eine rote LED. Lassen Sie diese wie bei einer Ampel abwechselnd leuchten. Nutzen Sie für die rote LED Port GPIO3 und für die gelbe LED Port GPIO2 wie in Abbildung 1.3 gezeigt.

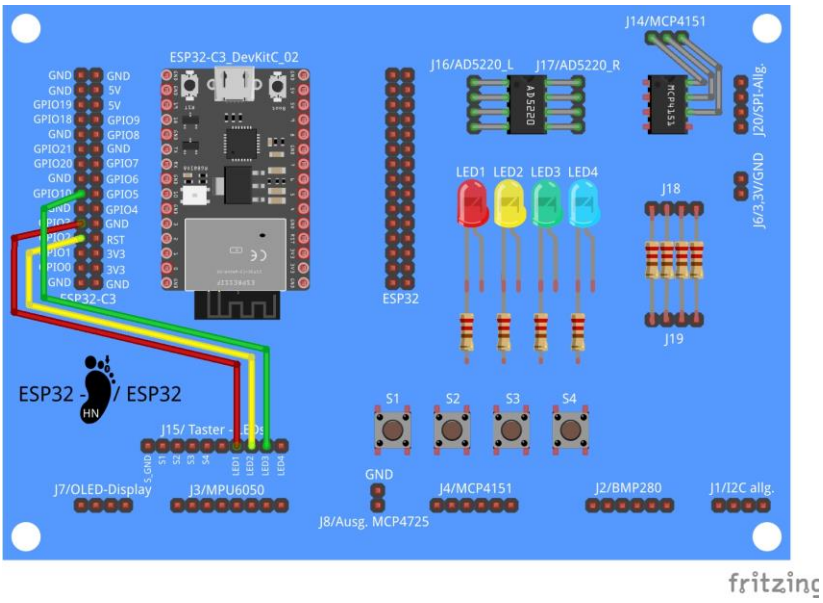


Abbildung 1.3: Verdrahtungsplan mit drei LEDs

2. Analoge Sensorwerte

In diesem Versuch werden wir Sensorwerte einlesen und über die serielle Schnittstelle auf dem PC darstellen. Lassen Sie die drei LEDs aus dem vorherigen Teil gesteckt und ergänzen Sie die Schaltung entsprechend Abbildung 2.1.

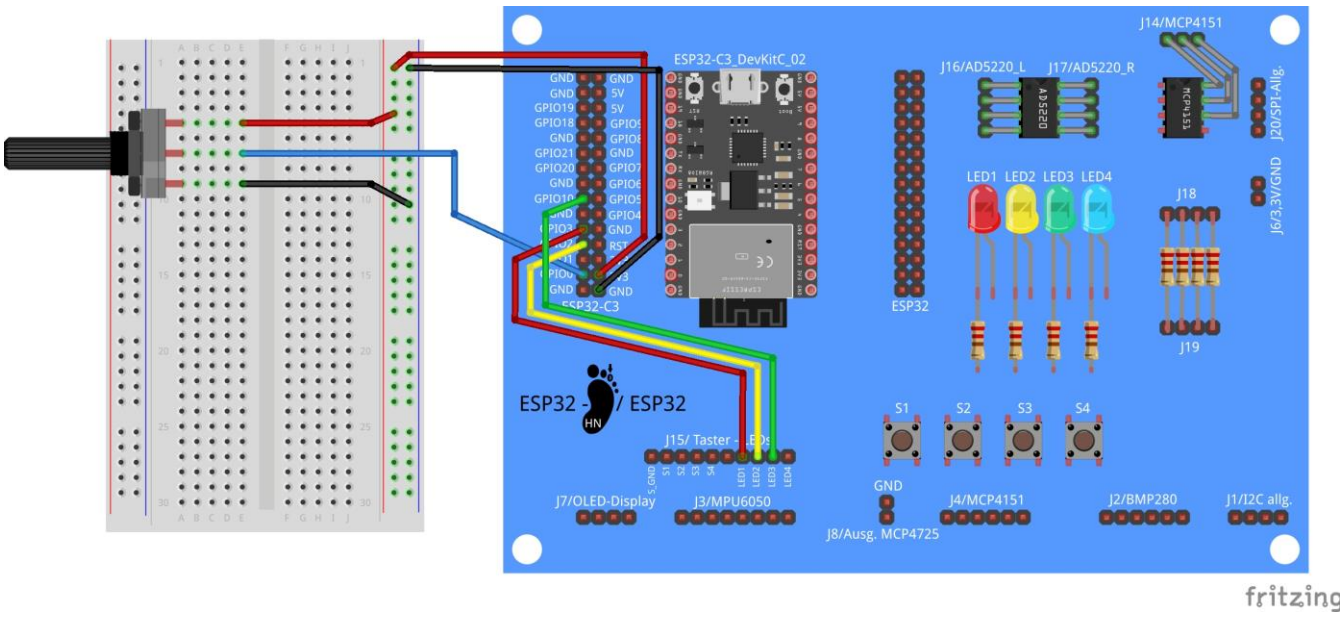


Abbildung 2.1: Verdrahtungsplan mit Potentiometer

Das neue Bauteil ist ein sogenanntes Potentiometer. Ein „Poti“ ist ein variabler Widerstand, dessen Widerstandswert sich durch drehen von 0Ω bis zum Maximalwert (hier: 10 kΩ) einstellen lässt. Wie bei fast allen Mikrocontrollern sind auch hier die einzelnen Kontakte des ESP32-C3 mit mehreren Funktionen belegt. Einen genaueren Überblick finden Sie in Abbildung 4.4 am Ende des Skriptes. Angeschlossen wird das Potentiometer an einen der 5 analogen Eingänge des ESP32-C3, in diesem Beispiel an Port GPIO0 (ADC1_CH0). Das Potentiometer steht beispielhaft für verschiedene Sensoren (Licht, Temperatur, Lautstärke, ...).

Mit dem folgenden Programm können Sie den Sensor auslesen:

```
/* Ein Potentiometer wird über den analogen Port ADC1_CH0 eingelesen */

int gruen = 10;
int gelb = 2;
int rot = 3;
int sensorwert = 0;
```

Testen

```
void setup()
{
        pinMode(gruen, OUTPUT);
        pinMode(gelb, OUTPUT);
        pinMode(rot, OUTPUT);
        Serial.begin(9600);
}

void loop()
{
        sensorwert = analogRead(A0);
        Serial.println(sensorwert);
        delay(1);
}
```

Nach dem Upload des Programms auf den ESP32-C3 passiert zunächst einmal nichts. Damit man sich die Werte ansehen kann, öffnet man unter dem Menüpunkt „Werkzeuge“ den „Serieller Monitor“, oder rechts oben in der Ecke der Arduino-  IDE den Button drücken. Jetzt sollten kontinuierlich Werte angezeigt werden. Da der Mikrocontroller über einen 12 Bit Analog/Digital-Wandler verfügt, werden Sie Werte zwischen 0 und 4095 (2^{12} = 4096) beobachten.

Aufgaben

- Erweitern Sie das Programm derart, dass mit dem Potentiometer die Blinkfrequenz einer oder mehrerer LEDs eingestellt werden kann. Ergänzen Sie das obige Programm dazu um die Ansteuerung der LEDs aus dem ersten Versuch.

- Alternativ können Sie den Sensorwert auch nutzen, um einzustellen, welche LED aktiv sein soll. Im unteren Drittel des Wertebereichs leuchtet beispielsweise die grüne LED, im mittleren die gelbe LED und im oberen die rote LED.



Abbildung 2.2: Einteilung des Wertebereiches

Das Codegerüst für eine dafür nötige Bedingung (if/then/else) sieht so aus:

```
if (sensorwert >= 0    &&    sensorwert < 341)
{
        digitalWrite(gruen, HIGH);
        digitalWrite(gelb, LOW);
        digitalWrite(rot, LOW);
```

Testen

```
}
else
{
        //dieser Code würde ausgeführt wenn die if-Bedingung nicht zutrifft.
}
```

Überlegen Sie wie viele if-Anweisungen Sie benötigen um den Bereich von 0 – 4095, wie in Abbildung 2.2 beschrieben, abzubilden.

3. Ultraschallentfernungsmessung

In diesem Versuch werden wir einen weiteren Sensor kennenlernen. Der HC-SR04 ist ein Ultraschallsensormodul, dass man sehr günstig(ca.3 €) im Internet erwerben kann. Mit ihm lässt sich die Laufzeit eines kurzen Schallimpulses bestimmen. Dies kann man dann leicht verwenden, um den Abstand des Moduls zum nächsten Hindernis zu berechnen. Diese Technik wird in der Praxis häufig eingesetzt. Beispielsweise funktionieren viele Rückfahrt-Abstandswarnsysteme in Autos nach diesem Prinzip.

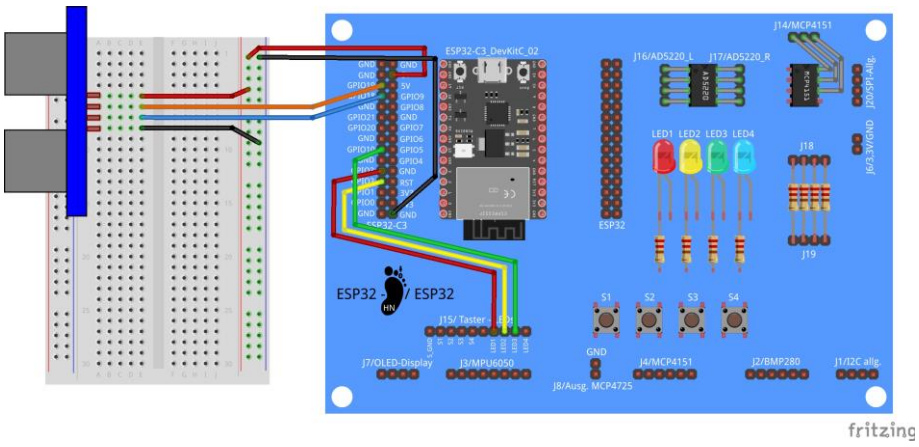


Abbildung 3.0-1: Verdrahtungsplan Ultraschallsensor und LEDs

Zunächst verkabeln wir das Modul wie in Abbildung 3.1 gezeigt. Die bestehende Schaltung für die LEDs aus den vorherigen Versuchen bleibt zusätzlich bestehen. Lassen Sie Ihren Aufbau anschließend von einem Betreuer prüfen.

Testen

Jetzt werden wir in der Arduino IDE ein Programm schreiben, dass uns die Entfernungswerte auf den PC Bildschirm ausgibt. Wie Sie sehen ist dieses Programm bereits geringfügig umfangreicher als die letzten Programme.

```
/* Ultraschall: Ein Ultraschallsensor liefert Werte zum PC.*/

int echoPin = 18;                       // Echo Pin
int trigPin = 19;                       // Trigger Pin
long laufzeit, entfernung;

void setup()
{
        Serial.begin (9600);
        pinMode(trigPin, OUTPUT);
        pinMode(echoPin, INPUT);
}

void loop()
{
        digitalWrite(trigPin, LOW);     // Trigger delayMicroseconds(2);
        digitalWrite(trigPin, HIGH);
        delayMicroseconds(10);
        digitalWrite(trigPin, LOW);
        laufzeit = pulseIn(echoPin, HIGH);
        entfernung = laufzeit*0.017;    // Berechnung der Distanz in cm.
        Serial.println(entfernung);
        delay(50); // 50 ms warten
}
```

Der Faktor 0.017:

Die Laufzeit wird in µs zurückgegeben.
Die Schallgeschwindigkeit in Luft (bei ca. 20°) beträgt rund 340m/s . Da Hin und Rückweg des Schalls gemessen werden, wird die Schallgeschwindigkeit noch durch 2 dividiert. Daraus ergibt sich: $0{,}017 = \frac{\frac{34000\ cm}{1000000\ \mu s}}{2}$

Aufgaben

- Halten Sie testweise Ihre Hand und andere Objekte in verschiedenen Entfernungen vor den Sensor, die Werte auf dem Bildschirm sollten entsprechend variieren.
- Stellen Sie das Messergebniss mittels LED(s) grafisch dar. Realisieren Sie dazu einen Schwellwertschalter, indem Sie eine LED leuchten lassen sofern der aktuelle Wert kleiner 30 cm beträgt. Nutzen Sie dafür wieder eine Bedingung (if/then/else).
- (optional:) Stellen Sie den Sensorwert mit den drei LEDs dar, wobei jede Farbe eine unterschiedliche Entfernung signalisiert (z.B. Rot -nah).

4. OLED-Display

Als nächstes widmen wir uns der Darstellung auf einem OLED-Display, wie Sie bereits in einigen Geräten zu finden sind. Zur Verfügung steht ein einfaches schwarz/weiß Modul mit einer Auflösung von 128 * 64 Pixeln. Für die Kommunikation verfügt das Display über eine I2C-Schnittstelle. Der I2C-Bus ist ein sehr verbreitetes Bussystem, welches bei vielen Sensoren zum Einsatz kommt.

Weiterführende Informationen zum I2C-Bus finden Sie unter
https://www.ipd.kit.edu/~buchmann/microcontroller/i2c.htm

Zunächst verbinden Sie das Display bitte wie in Abbildung 4.1 gezeigt mit dem ESP32-C3. Die nachfolgende Tabelle veranschaulicht nochmal die einzelnen Verbindungen die für den I2C-Bus benötigt werden.

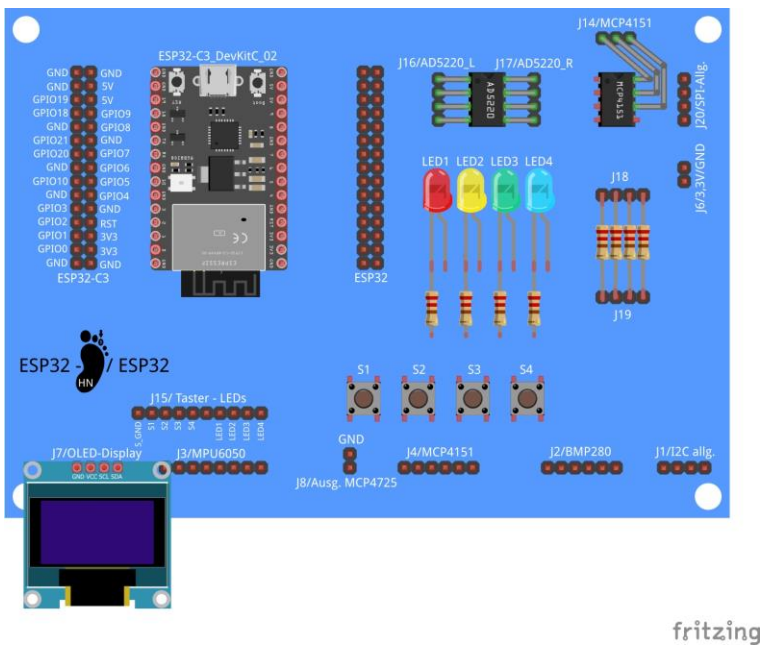


Abbildung 4.1: Verdrahtungsplan mit OLED-Display

OLED-Display	ESP32-C3
GND	GND
VCC	3V3
SCL	GPIO9
SDA	GPIO8

Testen

Anschließend geben Sie das folgende Programm ein:

```
/* OLED-Test: Gebe „Hello World!“ auf einem OLED-Display aus.*/

#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>

#define OLED_RESET 4 // not used / nicht genutzt bei diesem Display
Adafruit_SSD1306 display(OLED_RESET);

void setup()
{
        // initialize with the I2C addr 0x3C / mit I2C-Adresse 0x3c initialisieren
        display.begin(SSD1306_SWITCHCAPVCC, 0x3C);
}

void loop()
{
        display.clearDisplay();         // Display löschen
        display.setTextColor(WHITE);    // Textfarbe setzen
        display.setCursor(1,0);         // Textstartposition einstellen
        display.setTextSize(1);         // Textsize = 1 ➔ 6 * 8
        display.println("Hello World");
        display.setTextSize(2);         // Textsize = 1 ➔ 12 * 16
        display.println(millis());      //Ausgabe der Sekunden seit dem letzten Reset
        display.display();              //Ausgabe der übertragenen Daten auf dem Display
        delay(500);
}
```

Auf dem Display sollte nun der Schriftzug „Hello World!“ in der ersten Zeile und eine sich ändernde Sekundenanzeige sichtbar sein.

Aufgaben

- Modifizieren Sie das Programm, so dass es Ihren Namen oder einen anderen Text ausgibt.
- Nun soll der Schaltzustand von zwei Tastern auf dem Display aufgegeben werden. Hierzu ergänzen Sie die Verdrahtung wie nachfolgend in Abbildung 4.2: dargestellt.

Testen

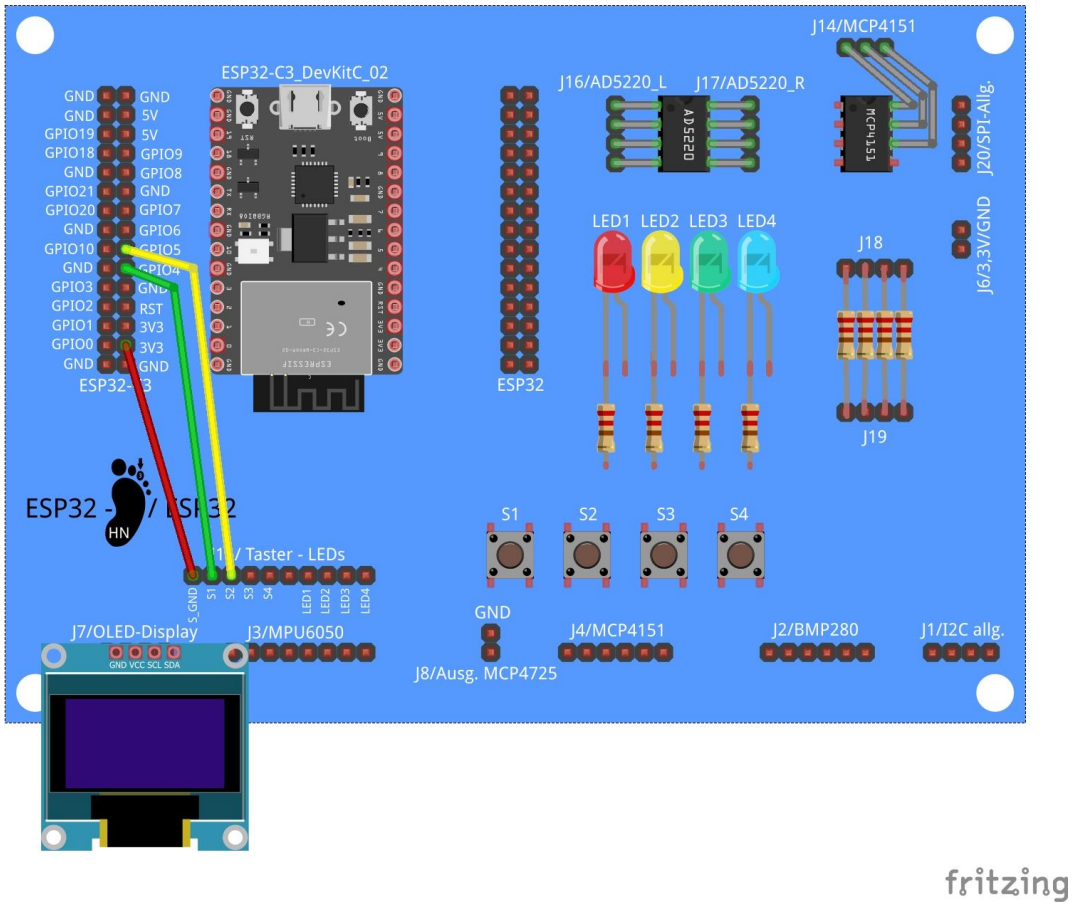


Abbildung 4.2: Verdrahtungsplan OLED mit Tastern

Die interne Verdrahtung der Taster und LEDs mit der Buchsenleiste J15 können Sie der nebenstehenden Abbildung 4.3 entnehmen. Die Eingangsseite der Taster können Sie über die Anschlüsse S1 – S4 der Buchsenleiste J15 abgreifen die Ausgangsseite der Taster liegt gemeinsam auf der Buchse „S_GND“. Somit kann man Signale sowohl gegen Masse (GND) als auch gegen eine Spannung (VCC, 3.3V oder 5V) schalten.

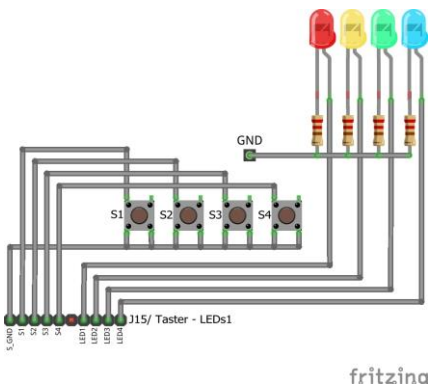


Abbildung 4.3: Anschlussbelegung der Buchsenleiste J15

Bei den LEDs sind die Anschlüsse der Widerstände zusammengelegt worden und mit dem GND-Anschluss des Mikrocontroller (ESP32-C3 oder ESP32) verbunden worden.

Testen

```
/* Sketch zum Testen eines OLED-Displays mit digitalem Input am ESP32-C3 */

#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>

#define OLED_RESET 4                    // nicht genutzt bei diesem Display
Adafruit_SSD1306 display(OLED_RESET);

int s1 = 5;                             // GPIO5
int s2 = 4;                             // GPIO4
String text_array[2] = { " geschlossen"," offen"};

void setup()
{
        display.begin(SSD1306_SWITCHCAPVCC, 0x3C);  // mit I2C-Adresse 0x3c initialisieren
        pinMode(s1, INPUT);                         // GPIO5 auf Eingang schalten
        pinMode(s2, INPUT);                         // GPIO4 auf Eingang schalten
}

void loop()
{
        int value_1= digitalRead(s1);
        int value_2= digitalRead(s2);

        display.clearDisplay();         // Display löschen
        display.setTextColor(WHITE);    // Textfarbe setzen
        display.setCursor(1,0);         // Textstartposition einstellen
        display.setTextSize(1);         // Textsize = 1 ( 6 * 8 )
        display.println("S1"+text_array[value_1]);
        display.println("S2"+text_array[value_2]);
        display.setTextSize(2);         // Textsize = 2 ( 12 * 16)
        display.println(millis());      //Ausgabe der Sekunden seit dem letzten Reset
        display.display();              //Ausgabe der übertragenen Daten auf dem Display
        delay(500);
}
```

Nach der Eingabe und dem Download des Programms sollten Sie auf dem Display den jeweiligen Schaltzustand der Taster S1 bzw. – S2 dargestellt bekommen.

Aufgaben

- Ändern Sie die delay(500) – Anweisung in delay(1), kompilieren und downloaden Sie die Software. Was können Sie beim Betätigen der Taster S1 und S2 auf dem Display beobachten?

Testen

Aus der so aufgebauten Schaltung kann man nun leicht die Funktion einer Stopp-Uhr erstellen. Dabei wird die Stopp-Uhr mit dem Button S1 gestartet und mit S2 wieder gestoppt. Die aktuelle Zeit wird dabei kontinuierlich auf dem Display ausgegeben.

Die *While-Schleife* ist dabei ein neues Element in der Software. Solange die Anweisung in runden Klammern hinter dem Schlüsselwort *While* wahr ist, werden die Anweisungen die sich in den darauffolgenden geschweiften Klammern {} befinden ausgeführt.

In der ersten While-Schleife wartet die Software auf die Betätigung von S1 (Start), in der zweiten darauf das S2 (Stopp) gedrückt wird.

```
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>

#define OLED_RESET 4 // not used / nicht genutzt bei diesem Display
Adafruit_SSD1306 display(OLED_RESET);

int s1 = 5;      // GPIO5
int s2 = 4;      // GPIO4
int value_1,value_2;
int count = 0;
String text_array[2] = { " geschlossen"," offen"};

void setup()
{
   display.begin(SSD1306_SWITCHCAPVCC, 0x3C);        //    mit I2C-Adresse 0x3c initialisieren
   pinMode(s1, INPUT);                               // GPIO5 auf Eingang schalten
   pinMode(s2, INPUT);                               // GPIO4 auf Eingang schalten

   display.setTextColor(WHITE);                      // Textfarbe setzen
   display.setTextSize(1);                                  // Textsize = 1 ( 6 * 8 )
}

void loop()
{
   display.clearDisplay();
   display.setCursor(1,0);
   display.println("Press S1 to start");
```

Testen

```
  display.display();

  while( digitalRead(s1) == 0)
  {
     delay(1);
  }

   int start = millis();

   while( digitalRead(s2) == 0)
  {
     display.clearDisplay();          // Display löschen
     display.setCursor(1,0);          // Textstartposition einstellen
     display.println("Press S2 to stop");
     display.println(float(millis() - start)/1000,4);      // float = Zahl mit Nachkommastellen / 4 =
                                                             Anzahl der Nachkommastellen
     display.display();
  }
  delay(10000);                              //Zeit zu Ablesen
}
```

Technische Daten OLED-Display 0,96“

Chipsatz	SSD1306
Farbe	Weiß
Sichtbarer Winkel	> 160°
Auflösung	128 x 64 Pixel
Betriebsspannung	3.3V bis 5V
Betriebstemperatur	-30°C bis +80°C
Niedriger Stromverbrauch	0,04W im normalen Betrieb
Ansteuerung	I2C / TWI
Pinbelegung	VCC, GND, SCL, SDA
Abmessungen	ca. 27mm x 27mm x 4,1mm

Optional - BMP 280

Um das Breakout-Board des BMP280 zu testen, stecken Sie bitte das Board wie in Abbildung 4.3 dargestellt auf Ihre Prozessor-Platine. Das hier abgebildete Model ist noch ein älteres mit 4 Anschlüssen. Ihr Board mit 6-Anschlüssen sollte den gesamten Stecker füllen. Achten Sie darauf dass die Anschlüsse VCC und GND auf dem Breakout-Board und der Prozessor-Platine über einstimmen. Der BMP280 ist nun mit dem ESP32-C3 über den I2C-Bus verbunden, und kann per Software angesprochen werden. Hierzu laden Sie bitte das Programm „BMP280test“. Sie finden das Programm unter dem Menü „Datei/ Beispiele/ Adafruit BMP280 Library /“.

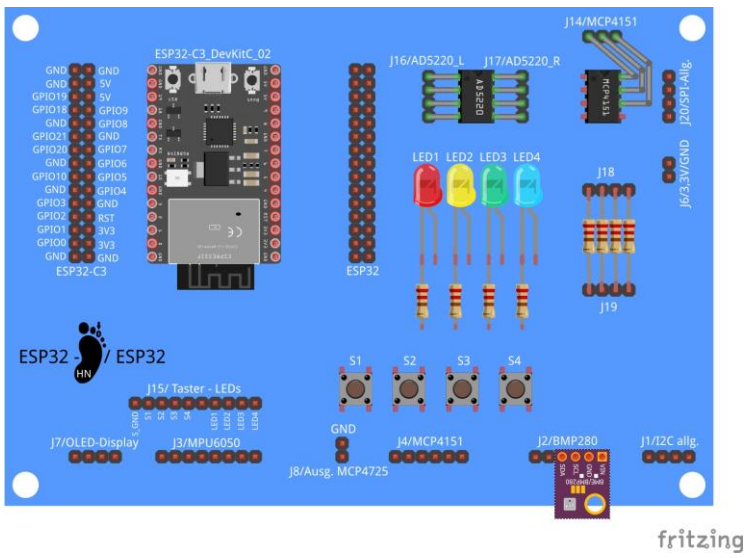


Abbildung 4.3: BMP280 - Breakout-Board

Bitte ändern Sie die folgende Zeile:

status = bmp.begin(); ➔ status = bmp.begin(0x76);

Die hex. Zahl 76 ist die Adresse des BMP280-Boards im I2C-Bus.

Kompilieren und downloaden Sie dieses Programm. Danach öffnen Sie bitte den seriellen Monitor. Hier sollten nun die Daten, wie nachstehend dargestellt, des BMP280 fortlaufend angezeigt werden.

Temperature = 26.02 *C
Pressure = 101028.28 Pa
Approx altitude = 24.73 m

Temperature = 26.06 *C
Pressure = 101026.78 Pa
Approx altitude = 24.86 m

Weiterführende Informationen

Funktionen der eingebundenen Library's

Class Reference	**Link**
Adafruit_SSD1306.h	https://adafruit.github.io/Adafruit_SSD1306/html/class_adafruit___s_s_d1306.html
Adafruit_GFX.h	http://adafruit.github.io/Adafruit-GFX-Library/html/class_adafruit___g_f_x.html
Adafruit BMP280.h	http://adafruit.github.io/Adafruit_BME280_Library/html/class_adafruit___b_m_e280.html
… weitere Bibliotheken für den ESP32…	https://www.arduinolibraries.info/architectures/esp32

Dieser Workshop dient als erster Einstieg in die Welt der ESP32-Familie. Wenn Sie sich in einem ähnlichen Stil weiter mit dem ESP32-C3 beschäftigen möchten, so finden Sie im Internet viele ausführliche Tutorials, beispielsweise bei Adafruit *(https://learn.adafruit.com)* ,SparkFun *(https://learn.sparkfun.com)*, oder auch Github (https://github.com/search?q=esp32-c3)

Eine wichtige Anlaufstelle für weitergehende Schritte ist natürlich auch die offizielle Seite *http://www.arduino.cc*. Dort kann man die Entwicklungsumgebung herunterladen und findet auch viel Dokumentation zur ESP32-Familie

Falls Sie sich bereits mit anderen Mikrocontrollern auskennen, hier einmal die Eckdaten des ESP32-C3:

Spezifikationen des ESP32-C3.

- RISC-V CPU mit 160 MHz
- 400 KB SRAM (16 KB als Flash Cache)
- Integriertes 2,4-GHz-WLAN (BGN)
- Bluetooth LE 5.0
- Kryptographische Hardware-Beschleunigung
- 22 programmierbare GPIOs
- 2 x 12-Bit-SAR ADC
- 3 x SPI (unterstützt SPI, Dual SPI, Quad SPI und QPI)
- 2 x UART (unterstützt RS232, RS485 IrDA mit bis zu 5 MBd)
- 1 x I2C (bis zu 800 kbit/s)
- 1 x I2S

- RMT (Fernsteuerungs-Peripheriegeräte)
- TWAI (CAN 2.0 b kompatibel / ISO 11898-1)
- PWM
- Interner USB/JTAG-Adapter

Pinout des ESP32-C3_DevKitC_02 Boards

